National Academy of Science and Technology

(Affiliated to Pokhara University)

Accredited by University Grand Commission (UGC), Nepal (2022)

Uttarbehadi-04 Dhangadhi , Kailali, Nepal

A

Project Report

On

Smart Study Planner

For the partial fulfillment of the requirements for the degree of Bachelor of Computer Engineering Under Pokhara University

Submitted to

Department of Computer Engineering

National Academy of Science and Technology

Under the Supervision of

Mr. Sunil Bahadur Bist

Lecturer, Department of Computer Engineering

Submitted by

Deepa Kumari Paneru (23070012)

Garima Thapa (23070018)

BE Computer, 6th Semester

August ,2026

DECLARATION

We hereby declare that this project report, entitled "Smart Study Planner", submitted to the Department of Computer Engineering, "National Academy of Science and Technology (NAST)", in partial fulfilment of the requirements for the award of the degree of Bachelor of Computer Engineering, is an original and authentic work carried out by us under the supervision of Mr. Sunil Bahadur Bist.

The work contained in this report has not been submitted in whole or in part to any other university or institution for the award of any degree or diploma. All information, data and findings presented are based on our own research and implementation. Where we have consulted the work of others, due acknowledgment has been given in the text and included in the list of references.

We understand that any instance of plagiarism or unauthorized use of others' work will be subject to the disciplinary actions of the university and academic regulations.

Submitted By:

……. **……..…….**

Deepa Kumari Paneru (23070012) Garima Thapa (23070018)

Date:

ACKNOWLEDGMENT

It is our pleasure to express our sincere gratitude to all those who have directly or indirectly contributed to the development of this project **Smart Study Planner** and who influence our thinking, behavior and acts during study. We are thankful for the valuable advice and suggestions provided by our teachers and friends during this project.

We express our sincere gratitude to **Mr. Sunil Bahadur Bist**, project supervisor, for providing essential suggestions, help and the solution to undergo final project. We are thankful to his support, cooperation and motivation provided to us during this first project for constant inspiration, presence, and blessings. We also obviously like to thank our team members: **Deepa Kumari Paneru, Garima Thapa.**

Finally, we are thankful to our respected **Principal Prof. Dr. Prem Bahadur Chand, Head of Department, Department of Computer Engineering, Er. Puspa Raj Joshi**, other Lecturers, friends, and who are involved to our projects as the medium of directness.

Project Team:

Deepa Kumari Paneru (23070012)

Garima Thapa (23070018)

SUPERVISOR ACCEPTANCE

This is to certify that the project entitled "Smart Study Planner" has been carried out by the team member Deepa Kumari Paneru (23070012 and Garima Thapa (23070018) under my supervisor in partial fulfillment of requirements for the degree of Bachelor of Computer Engineering accepted for the final defense.

To the Best of My Knowledge, this project is an original work completed by the students.

Supervisor
Mr. Sunil Bahadur Bist
Department of Computer Engineering
National Academy of Science and Technology

Signature: .....

Date: .....

ABSTRACT

A **Smart Study Planner** is a web-based application that provides a centralized platform for engineering students to register, log in, and manage their academic study sessions. The system includes a personalized dashboard, subject-wise progress tracking, a study task manager, a personal notes module, and a weekly study planner. It replaces the traditional manual study plans in a paper or notebook.

On the administrative side, the system enables administrators to manage notes, student information, weekly plans, subject records through a secure dashboard.

Overall, this project demonstrates how modern web technologies can be used to smart study, making the study of students more efficient. It provides a notification to students when their deadline of plans comes near and the plan isn't complete. It shows the study status of students by visual way i.e. bar graph. Mobile applications, Student reviews, and AI-based study recommendations in the future.

***Keywords*: Smart Study Planner, Weekly plan, Notes Management, Study Streak.**

LIST OF TABLES

Table 3.2.3: Requirement Prioritization Table…….7

***Table 3.3.1: Cost Scheduling*.....8**

Table 3.3.2 Task Scheduling…….9

Table 5.5 Test Cases……21

Table 6.1 Test Report……..23

L I S T OF FIGURES

Figure 4.1: Incremental Model ..... ……..……..11

Figure 4.2.1: Architecture Design..... 13

Figure 4.3.1: Use case Diagram..... 14

Fig 4.3.2.1: Level 0 Data Flow Diagram 15

Fig 4.3.2.2: Level 1Data Flow Diagram 16

Fig 4.4: Schema Diagram..... 16

Table of Contents

DECLARATION……...i

DECLARATION i

ACKNOWLEDGMENT ii

SUPERVISOR ACCEPTANCE iii

ABSTRACT iv

Keywords. iv

LIST OF TABLES v

L I S T OF FIGURES vi

LIST OF ABBREVIATIONS xi

CHAPTER 1 1

INTRODUCTION 1

1.1 Background 1

1.2 Objectives 1

1.3 Purpose, Scope and Applicability 2

1.3.1Purpose 2

1.3.2 Scope 2

1.3.3Applicability 2

1.4 Achievement 2

1.5 Organization of the report 3

CHAPTER 2 4

SURVEY OF RELATED WORKS 4

2.1 Related Work Review 4

2.1.1 Global E-Learning Platforms and LMS 4

2.1.2 Localized and Niche Study Trackers 4

2.1.3 Digital Scheduling Tools *4*

2.2 Gaps in the Existing System 5

CHAPTER 3 6

REQUIREMENT AND ANALYSIS 6

3.1 Problem Definition 6

3.2 Requirement Specifications 6

3.2.1 Functional Requirements *6*

3.2.2 Non-Functional Requirements *6*

3.3 Planning and Scheduling 7

3.3.1 Cost Scheduling *7*

3.3.2 Task Scheduling *9*

3.4 Software and Hardware Requirements 10

3.5 Preliminary Product Description 10

3.6 Conceptual Models 11

CHAPTER 4 12

DESIGN 12

4.1 Introduction 12

4.2System Design 13

4.3Interface Design 14

4.3.1Use Case Diagram *14*

4.3.2 Data Flow Diagram *15*

4.4 Database Design 16

4.5 Summary 17

CHAPTER 5 18

IMPLEMENTATION AND TESTING 18

5.1 Implementation Approaches 18

5.2 Coding Details and Code Efficiency 18

5.2.1Code Efficiency 18

5.3Testing Approach 19

5.3.1 Unit Testing ***19***

5.3.2 Integrating Testing ***19***

5.3.3 Beta Testing ***20***

5.4 Modifications and Improvements 20

5.5 Test Cases 21

6.2 User Documentation 25

CHAPTER 7 26

CONCLUSION AND RECOMMENDATIONS 26

7.1Conclusion 26

7.1.1 Significance of the System ***26***

7.1.2 Limitations of the System ***26***

7.2 Recommendations 26

ANNEX I 28

Screenshots: 28

..... 29

ANNEX II 30

Source code 30

ANNEX III 35

Test Cases source code 35

LIST OF ABBREVIATIONS

API: - Application Programming Interface

BE: - Bachelor’s in engineering

CRUD: Create Read Update and Delete

DB: - Database

DFD: - Data Flow Diagram

Er: - Engineer

ER-Diagram: - Entity Relationship Diagram

FAQ: - Frequently Asked Question

FAQ: - Frequently Asked Question

GB: - Gigabyte

GUI: - Graphical User Interface

HOD: - Head of Department

IDE: Integrated Development Environment

JSON: JavaScript Object Notation

RDBMS: - Relational Database Management System

SQL: - Structure Query Language

UI: User Interface

UML: - Unified Modeling Language

CHAPTER 1

INTRODUCTION

1.1 Background

In today's competitive academic environment, engineering students in Nepal are expected to manage multiple subjects, laboratory practical's, assignments, and examinations simultaneously across a single semester. Tribhuvan University and Pokhara University affiliated engineering colleges require students to maintain consistent academic performance across six to seven subjects every semester. Despite this pressure, most students lack a structured and digital method to plan and track their study sessions effectively. Traditional study planning methods such as physical notebooks, paper timetables, or general-purpose apps like Google Calendar are either too generic or not accessible without internet. They offer no subject-specific progress tracking, no automatic study hour tallying, and no visual overview of how well a student is prepared across all subjects.

This project proposes the development of a **Web-Based Smart Study Plan System** that helps the students to study smartly. The system will allow students to upload or write notes, weekly plan and access anywhere at any time offline or online. On the other hand, the smart study planner admin will be able to efficiently manage notes, subjects, students, and plans through a centralized dashboard. By digitizing the study system, this system aims to reduce manual work, helping student's study more smartly. It provides notification to meet deadlines of weekly plan.

1.2 Objectives

The main objective of the Smart Study Planner is to develop a secure, student-friendly, and reliable platform that enables students to add and access notes, make plans while allowing owners and administrators to efficiently manage study works digitally.

1.3 Purpose, Scope and Applicability

1.3.1Purpose

The purpose of this is to design and develop Smart Study system, a digital platform that helps engineering students plan, track, and review their academic study activities in an organized and efficient manner.

The system is intended to manage students' information, notes, and plans added by students automatically create study streaks and notify students that motivate students to read on time. It helps to manage pressure of engineering students and promote smart study practices.

1.3.2 Scope

The system Can be used for:

- Note uploading and writing
- Subjects and note management
- Student management
- Study Planning
- Study Report generation

1.3.3Applicability

The **Smart Study System** is applicable to students and candidates preparing for competiti*ve exams*. provides an online platform for students to search, add, and manage study plans efficiently. The system automatically notifies students for study within limits of time It improves study efficiency, enhances students' satisfaction through digital study plans, and enables students to manage their study more effectively.

1.4 Achievement

The Smart Study **System** successfully digitalizes the traditional manual paper-based study plans by providing an efficient and student friendly digital platform. It simplifies note adding, weekly plan, and student management while reducing paperwork and manual errors. The

system offers notes availability, secure user authentication, and an easy-to-use administrative dashboard for managing notes and plans. It improves operational efficiency, saves time for students and administrators, enhances data accuracy, and provides a scalable foundation for future enhancements such as mobile application support and recommendation.

1.5 Organization of the report

This report is organized into five chapters, each covering a different stage of the development of the **Smart Study Planner**. Chapter 1: Introduction introduces the Smart Study Planner and presents its background, objectives, purpose, scope, applicability, achievements, and overall organization of the report. Chapter 2: Literature Review and Tools & Technologies reviews related study plans platforms and discusses the tools and technologies used in the development of the system, including HTML, CSS, JavaScript, Python, Flask, MySQL, SQL Alchemy, and Visual Studio Code. Chapter 3: Requirement Analysis defines the problem statement and presents the functional and non-functional requirements, project planning and scheduling, hardware and software requirements, preliminary product description, and conceptual models. Chapter 4: System Design describes the overall system architecture, interface design, use case diagram, data flow diagram, database design, and other design components that illustrate the structure and operation of the proposed system. Chapter 5: Conclusion and Recommendations summarize the achievements and significance of the Smart Study Planner and provides recommendations for future improvements and enhancements.

CHAPTER 2

SURVEY OF RELATED WORKS

2.1 Related Work Review

In developing our system, we reviewed several existing digital education, management, and planning platforms. These systems can be broadly categorized into E-learning platforms, Learning Management Systems (LMS), and digital scheduling tools.

2.1.1 Global E-Learning Platforms and LMS

- **Coursera:** This is a massive open online course (MOOC) provider that offers high-quality educational content from top universities. While it is excellent for self-paced learning, it operates on a global scale and requires expensive premium subscriptions for certificates. It does not provide daily localized study planning or offline features for students.
- **Sakai:** This is an open-source Learning Management System used by universities to distribute course materials, grades, and assignments. However, Sakai is heavily institution dependent. If a school is not set up, individual students cannot use it to manage their personal study routines.

2.1.2 Localized and Niche Study Trackers

- Chautari & Study Track: These platforms offer tailored solutions for student engagement and progress tracking. They help students organize tasks and follow specific educational paths. However, like global systems, they rely completely on a stable, high-speed internet connection and are built primarily for desktop users rather than mobile-first environments.

2.1.3 Digital Scheduling Tools

- **Google Calendar:** This is a widely used time-management and scheduling tool. Students can use it to block out study times and set reminders. However, Google Calendar is a general-purpose tool. It lacks specific student-centric features, such as note-taking capabilities, progress tracking, and syllabus management.

2.2 Gaps in the Existing System

Despite the availability of these global and local digital systems, a significant gap exists in Nepal's education ecosystem:

Internet Dependency: Most existing tools are strictly cloud-based. In Nepal, frequent internet disruptions and the high cost of mobile data make it difficult for students to stay connected 24/7.

Subscription Barriers: Platforms like Coursera require payment methods (like international credit cards) that are not easily accessible to the average Nepali student.

The Paper Dependency: Because of these digital barriers, most Nepali students still rely on traditional paper-based planners, physical notebooks, and handwritten diaries. These physical methods are easily lost, hard to back up, and cannot provide smart reminders.

Weekly Plan: No week plan as user chooses own note.

CHAPTER 3

REQUIREMENT AND ANALYSIS

3.1 Problem Definition

Current Problem and Proposed Solution

1.To study Smartly using digital platform is rare for engineering students. As they have a lot of loads a pressure just depend on paper plans.

2.Existing online Study plan System are also not fully effective for students as they require internet and sometimes subscriptions. Platforms like Google Calendar and Coursera often provide planning but require internet. These limitations highlight the need for a more affordable, location-based, and efficient vehicle rental system designed specifically for local users and rental businesses.

3.To address these challenges, we propose a web-based Smart Study Planner to help to study, adding notes and giving notification without require internet.

3.2Requirement Specifications

3.2.1 Functional Requirements

a) User login and Signup

b) Admin dashboard for managing system

c) Subject Manager

d) Student dashboard

e) Note module

f) Weekly planner

g) Study Progress Tracker

3.2.2 Non-Functional Requirements

a) Secure authentication

b) Responsive frontend interface

c) Modular code structure

d) Systems should load pages within two seconds on a standard laptop.

e) All data must be scoped to the logged-in user

3.2 .3 Requirement Prioritization Table

Requirement	**Priority level**	**Description**
User Login & signup (student or admin)	1	Core authentication is required before accessing any features
Subject Manager	2	Required to organize tasks and notes by subject
Note Module	3	Allows students to write and retrieve personal study notes
Weekly Planner	4	Helps students schedule sessions across the week
Study Tracker Streak	5	Motivational feature, tracks consecutive study
Student Dashboard 6	6	Gives students a real-time overview of their study progress, streaks, and quick links.
Admin Dashboard	7	Secure administrative panel to manage overall system and global data.

Table 3.2.3: Requirement Prioritization Table

3.3 Planning and Scheduling

Planning and Scheduling consist of cost scheduling task scheduling and time scheduling.

3.3.1 Cost Scheduling

The development cost of this proposed project will be estimated as:

SN	**Items**	**Details**			**Estimated Cost (NRs)**
1	Development Tools	Python, Flask, MySQL, XAMPP			Open source
2	Hardware Depreciation	Laptop depreciation charge for 2.3 months			60000
3	Documentation and Printing	Proposal, Project Report, technical documentation			1000
4	Development Cost	Module	Estimated hour	Estimated cost (NRs)	49600
		Authentication Module	8hr	4800	
		Subject Module	5hr	4000	
		Note Module	12hr	8400	
		Weekly Planner Module	12hr	8400	
		Study Streak Module	10hr	8000	
		Student Module	10hr	8000	
		Admin Module	10hr	8000	
5	Miscellaneous Cost	Rent, internet, electricity, snacks, stationery, testing etc.			40000

Total Cost	89,600

Table 3.3.1: Cost Scheduling

3.3.2 Task Scheduling

This project schedule has been designed based on the requirements and constraints involved. The project is planned to begin in the second week of May 2026 and is scheduled for completion by the second week of August 2026, making the total duration approximately 3 months. In this feasibility study, we analyze the time required for each phase of the project, including analysis, planning, designing, coding, testing, documentation, and final presentation

The project work is divided into various activities which will be carried out as per given schedule:

Activities	**May**		**June**				**July**				**August**	
	W3	**W4**	**W1**	**W2**	**W3**	**W4**	**W1**	**W2**	**W3**	**W4**	**W1**	**W2**
Problem Identification												
Requirement Analysis												
Feasibility Study												
Designing												
Proposal Submission												
Proposal Defense												
Coding												
Mid-term Defense												
Testing												
Report Submission												
Final Defense												

Feedback												
Documentation												

Table 3.3.2: Task Scheduling

3.4 Software and Hardware Requirements

Hardware Requirements

- Computer or laptop
- RAM (Minimum 8 GB or Above), Disk (Minimum 100 GB or Above)
- Stable Internet Connection

Software Requirements

- VS Code
- Python 3.14
- Operating System windows 10 or above

3.5 Preliminary Product Description

This system offers:

- A web application for Administrators and Students to manage user accounts, notes adding, accessing, weekly plan, and notes management.
- A centralized platform that enables students to show their study status graphically.
- A student -friendly interface **for s**tudy **to** browse available notes, plans, subjects.
- System provides automatically notifications for deadline meet.

3.6 Conceptual Models

- **Administrator:** Manages users, notes, and overall system operations. Monitors system activities, generates study status reports, and ensures the smooth and secure functioning of the platform.

- **Students:** Adds subjects wise notes, weekly plan, access notes and study according to plans

CHAPTER 4

DESIGN

4.1 Introduction

We have adopted the Incremental Model for the development of the Smart Study Planner This model allows the system to be built phase by phase, where each phase delivers a working application, continuously improves the system by incorporating regular feedback, testing, and reviews throughout the development process.

The Incremental Model is suitable for this project because the Smart Study Planner contains multiple modules such as user management, notes management, weekly plan, study streak, and reporting. Incremental mode allows these modules to be developed and tested incrementally, reducing development risk and improving overall system quality.

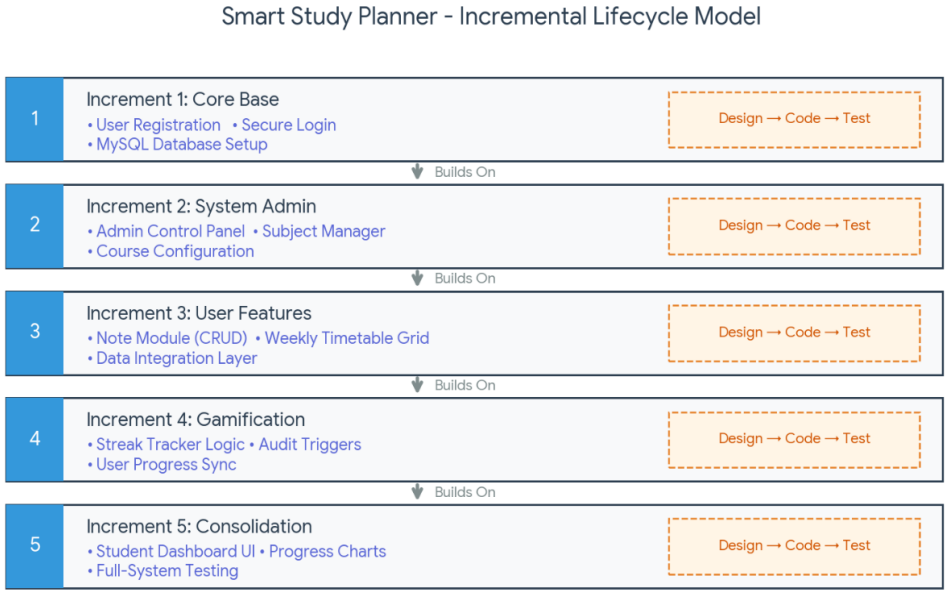


Fig 4.1: Incremental Model

4.2System Design

4.2.1Architectural Design

1. **The Smart Study Planner** is designed with a user-friendly interface, a secure backend, and a centralized database. The system is divided into two main parts: User Module and Admin Module.
2. **The Smart Study Planner** includes notes searching, adding, plans weekly features. The User Module includes adding, deleting and updating notes by subjects and access them, adding weekly plans, and a study status show. The **Admin Module** provides complete control over students, notes management, study streak and report generation.
3. The system uses a relational database (MySQL) to store all information securely. It features real-time notes availability checking, and automated generations of study status. The design focuses on simplicity, security, and responsiveness so that the system works smoothly on both computers and mobile devices.

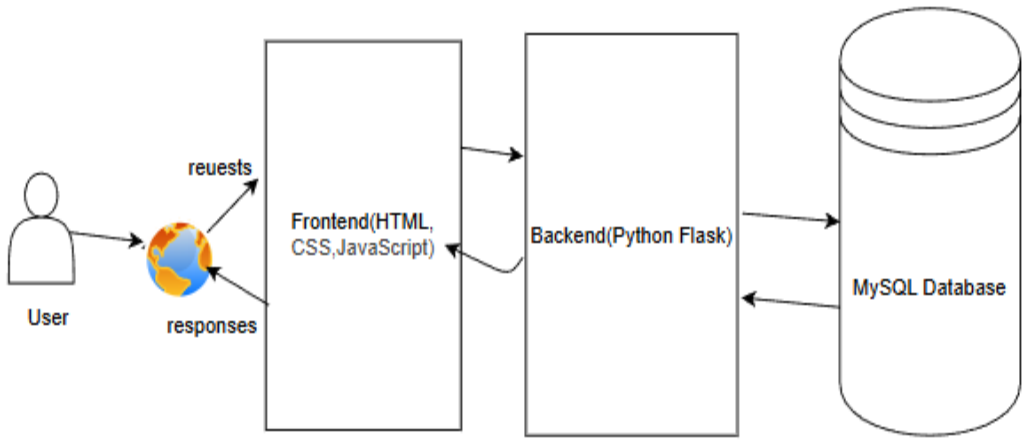


Figure4. 2.1: Architectural design

The system architecture is based on a client-server model, where users such as Admin and Student interact with the system through the internet using a web application or an Android mobile application. User requests such as login, accessing notes, adding notes and weekly plan, and study status tracking are sent to the server for processing. The requests are handled by the Web Server, which hosts a REST API server developed using Spring Boot. The API acts as the middle layer, managing core study logic including user authentication, student profile management, notes and plan, study status, study streak, and report generation, while ensuring secure communication between the frontend and backend. The backend is supported by a

MySQL database, which stores all essential data such as user details, student profile, notes, plans, and study status. Based on user actions, the server retrieves or updates data in the database and sends appropriate responses back to the client application

4.3Interface Design

4.3.1Use Case Diagram

A Use Case Diagram is a vital tool in system design it provides a visual representation of how users interact with a system. It serves as a blueprint for understanding the functional requirements of a system from a user's perspective, aiding in communication between stakeholders and guiding the development process. In our system the Use case diagram is designed as given below.

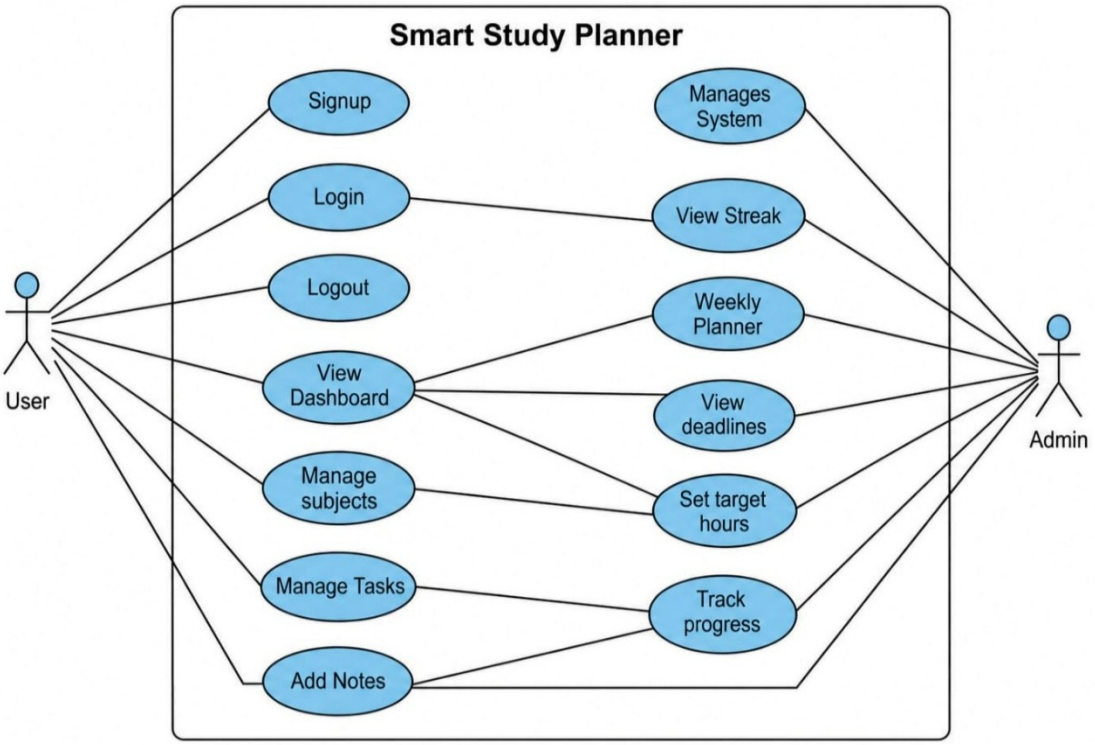


Figure 4.3.1: Use case Diagram

4.3.2 Data Flow Diagram

A DFD is a graphical representation that depicts the flow of data within a system.

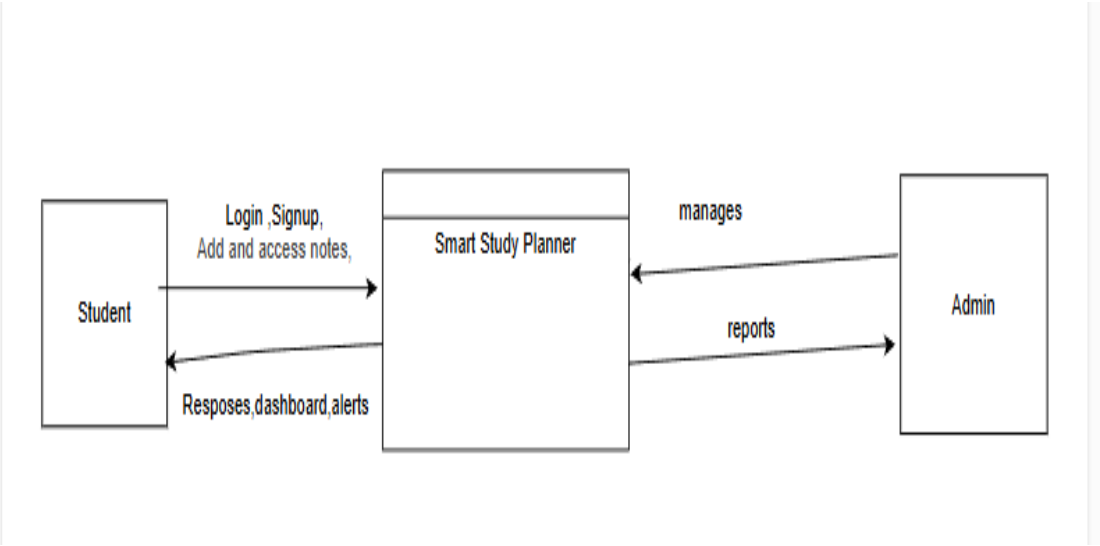


Fig 4.3.2.1: Level 0 Data Flow Diagram

The above figure illustrates the Level 0 Data Flow Diagram (DFD) of the Smart Study Planner which provides a high-level overview of how different entities interact with the system. The central process, Smart Study Planner labeled as, acts as the main processing unit where all requests and responses are handled.

The admin interacts with the system to manage users, notes, plans and overall system settings as well as to monitor platform activities. The student interacts with the system to add and update notes, plans, view notes, plans. This level 0 DFD shows only the major external entities and their data interactions with the Smart Study System without detailing internal processes, thereby providing a clear and simplified overview of the system's overall functionality.

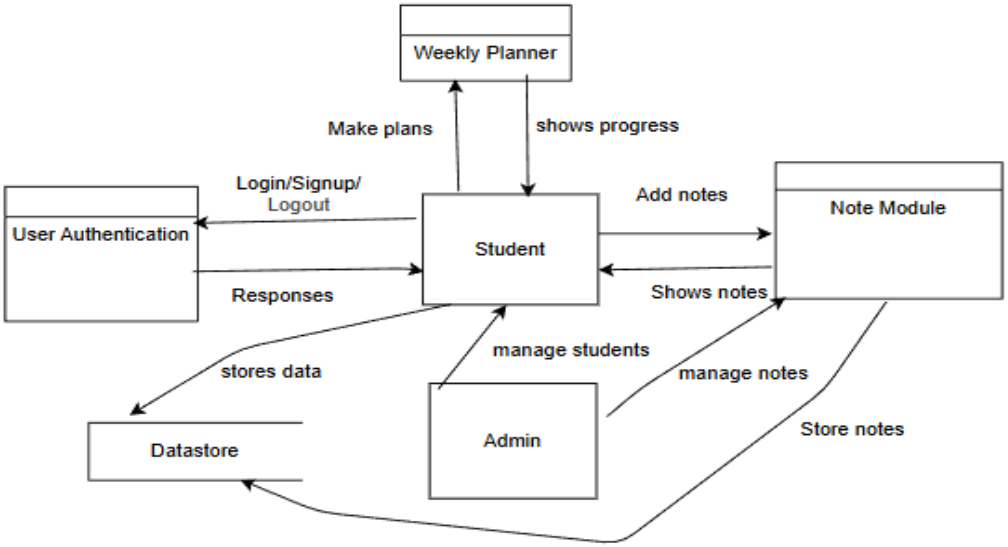


Fig 4.3.3.2: Level 1 Data Flow Diagram

4.4 Database Design

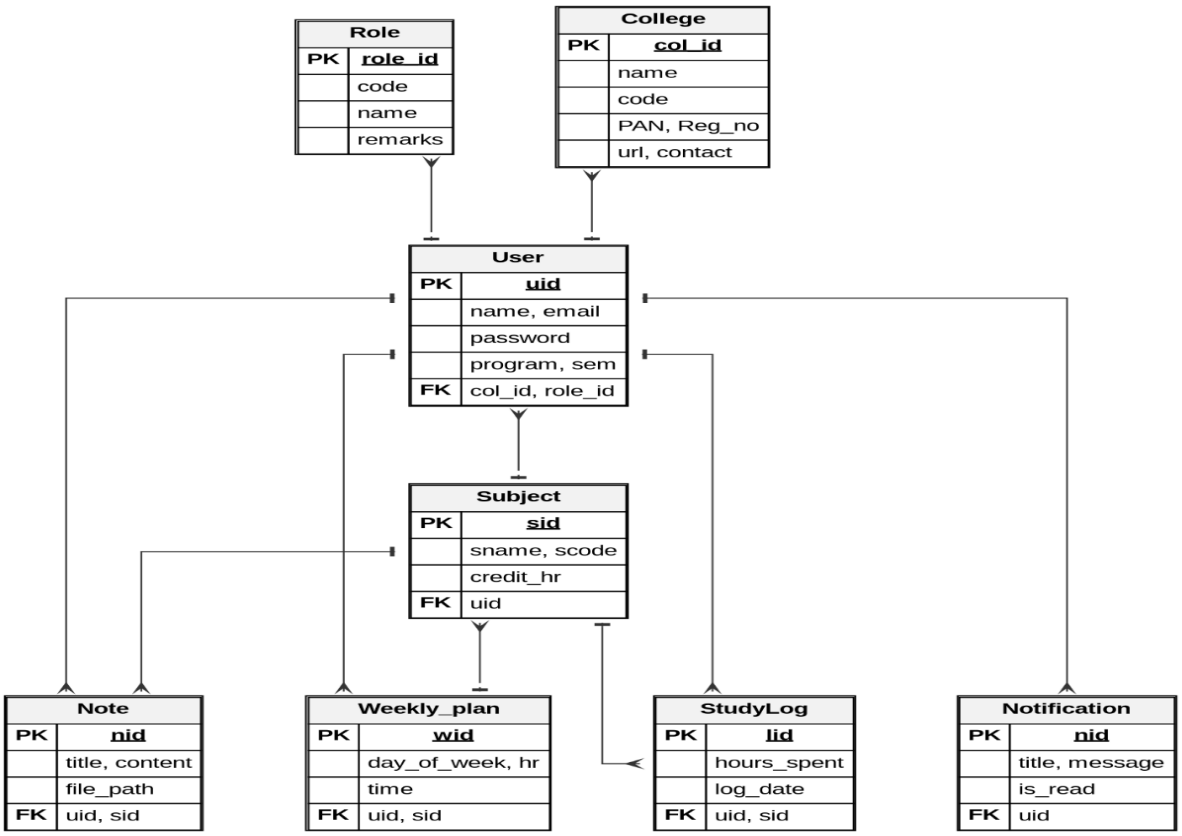


Fig 4.4: Schema Diagram

4.5: Summary

In Chapter 4, the overall design of the Smart Study Planner is presented. The system follows a structured architecture that separates the presentation layer, database layer, ensuring modularity, maintainability, security, and scalability. The database is designed to efficiently manage essential data, including user information, notes details, planning, reviews, and study status, enabling accurate and reliable processing of study activities.

The interface design is represented through UML diagrams such as use case diagrams, Data Flow diagrams, illustrating the interactions among different users, including the admin and students. These interactions include user authentication, note adding, note search, plans management and administrative operations.

The system design emphasizes secure user authentication, efficient workflow management, responsive user interfaces, and seamless communication between different system components. Overall, the proposed design provides a robust, organized, and user-friendly platform that simplifies the process of note adding and planning for student's enhancements and scalability.

CHAPTER 5

IMPLEMENTATION AND TESTING

5.1 Implementation Approaches

The implementation stage represents the phase where the designed Smart Study Planner System is converted into a working software prototype. The system is currently user active development, with core modules being implemented and integrated incrementally.

Currently, the backend services using Flask, the database using MySQL, and the basic frontend interfaces are developed. The primary focus of this stage is to ensure that the basic system structure, database connectivity, and core functionalities work correctly and reliably.

The major outcomes of this stage include:

- A working prototype of the system with basic modules such as user authentication, Student profile management, Note, plan adding and progress status show, notifications.
- A configured database with sample data for testing and development purposes.
- Source code management using a version control system to track development progress and changes.
- Basic integration between frontend, backend, and database to support end-to-end system functionality.

5.2 Coding Details and Code Efficiency

While developing the Smart Study Planner, special attention is being given to maintaining good code quality and efficiency. The system is being implemented using a modular and layered structure so that each part of the system has a clear responsibility. This makes the code easier to understand, test, and modify.

5.2.1Code Efficiency

Coding refers to the process of writing instructions using programming languages to develop an application or software system. In the Smart Study Planner, the implementation is being

carried out using well-known and reliable technologies. The focus of the code efficiency phase is to ensure that the system is efficient, maintainable, and scalable.

The code efficiency of the system is being ensured through the following practices:

- Writing clean, modular, and well-documented code so that each module has a clear responsibility and can be easily maintained or modified in the future.
- Using a layered and structured programming approach to separate user interface, business logic, and database operations.
- Reducing code duplication by using reusable classes, functions, and services wherever possible.
- Implementing proper input validation and exception handling to prevent system crashes and reduce logical errors.
- Organizing the source code properly and adding meaningful comments to make future debugging and enhancements easier.

5.3Testing Approach

5.3.1 Unit Testing

Unit testing was carried out to ensure that each module of the Smart Study Planner System functions correctly when tested independently. The modules tested include user login, OTP for password recovery, admin dashboard, Student profile management, note, plan management, update notes, plans, send notifications. For example, the login module was tested using both valid and invalid credentials to verify authentication and error handling. The OTP generation and verification module was tested to ensure secure and reliable password recovery. Note and Plan add and order-related modules were tested to confirm correct data input, validation, and retrieval from the database.

5.3.2 Integrating Testing

Integration testing was performed to ensure that all modules of the Smart Study Planner system work together smoothly and consistently. For example, notes added or updated by students through the note management module were correctly reflected in the note listing. Plans added

by customers were accurately displayed in the Weekly management System Changes made in the admin dashboard, such as updates to user roles, system settings, were verified to ensure that all related modules were updated consistently across the platform. The interaction between the login module and role-based dashboards (Admin, Student) was also tested to confirm proper authentication, authorization, and redirection.

5.3.3 Beta Testing

Beta testing is a crucial phase in software development where a nearly complete version of the application is released to a select group of real users, such as student to evaluate its performance in real-world conditions. Unlike earlier testing stages, beta testing occurs outside the development environment, providing insights into how the system functions under actual usage patterns. This phase is instrumental in identifying usability issues, performance bottlenecks, and edge cases that may not have been anticipated during internal testing. For instance, users might encounter interface problems on specific devices, experience slow load times during peak usage, or discover rare bugs triggered by unusual input combinations. By gathering and analyzing this feedback, developers can make informed adjustments to enhance the application's reliability and user experience before its official launch.

5.4 Modifications and Improvements

As the development and testing phases progress, several areas for improvement have been identified to further enhance the quality, usability and reliability of the Smart Study Planner System. These improvements aim to provide better user experience and more efficient system performance.

Refine User Interface and Workflows: Feedback from users such as Students, and administrators helps identify areas where the user interface and workflows can be improved. Simplifying note modules, plan module, and making dashboard navigation more intuitive can significantly enhance usability.

Fix Errors and Logical Issues: During testing minor bugs or logical issues may arise in modules such as note adding, plan management, notifications, or progress status updates. These issues are addressed to ensure accurate order processing and consistent system behavior.

Improve System Performance: System performance can be improved by optimizing database queries, improving backend logic, and refining data handling mechanisms. This ensures faster note, plan loading, notifications, and smooth operation even with many users and transactions.

Enhance Security and Data Reliability: Additional input validation, authentication checks, and role-based access control mechanisms can be implemented to prevent unauthorized access and protect sensitive user and transaction data.

Add New Features if Required: Based on future requirements, features such as Study Recommendation, mobile based, real-time notification, and SMS/mail order alerts can be added to increase the overall usefulness and impact of the Smart Study System.

5.5 Test Cases

In the Smart Study Planner system, effective testing is essential to ensure accurate note, plan management, secure access, and smooth system operation. Below are some sample test cases across various modules:

Test Cases	Test Type	Descriptions
1.	Unit Testing	Enter valid username and password and verify successful login and redirection to the dashboard.
2.	Unit Testing	Enter invalid credentials and confirm that an appropriate error message is displayed.
3.	U**nit Testing**	Token, Refresh Token, User ID and User Type are received from server when login is successful.
4.	U**nit Testing**	Login with Refresh Token, Token and User ID.
5.	Unit Testing	Request OTP for password reset and verify that it is sent to the registered email.
6.	Unit Testing	Enter the correct OTP and confirm successful password reset.
7.	Unit Testing	Add a note or plan and view it is saved to database

8.	Unit Testing	Add study session and see study progress via graphically.
9.	Integration Testing	Log in as admin and confirm that the dashboard loads with access to subjects, notes, plans, alerts
10.	Beta Testing	The system loads and displays all relevant user details on the profile page without errors or missing information. Only the user who added notes, plans can get his/her added notes, plans no other user can get it

Table 5.5: Test Cases

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Test Reports

The testing phase of the Smart Study Planner System ensures that the software is reliable, accurate, and aligned with the intended functional requirements of the platform. Although formal automated testing tools have not yet been implemented, manual testing has been carried out to validate the core functionalities of the system. This preliminary testing focused on critical modules such as user authentication, note management, planing, and dashboard operations to ensure correct behavior and data consistency. The outcomes of these tests have been recorded in test reports which provide a clear understanding of the system's current performance, stability, and reliability.

Test ID	Test Type	Descriptions	Expected Result	Actual Result	Status
1.	Unit Testing	Enter valid username and password.	Successful login and redirection to the dashboard.	Logged in successfully and redirected to the dashboard.	Passed
2.	Unit Testing	Enter invalid credentials.	error message display	Error message "Invalid username or password" was displayed.	Passed
3.	Unit Testing	Request OTP for password reset.	OTP is sent to the email.	OTP received by mail	Passed

4.	Unit Testing	Enter the correct OTP.	Confirm successful password reset.	Password updated successfully; user can log in with new password.	Passed
5.	Unit Testing	Add a note or plan.	View it and verify it is saved to the database.	Note and plan store din database	Passed
6.	Unit Testing	Add a study session.	See study progress updated graphically.	Progress chart and graphs updated with added session	passed
7.	Integration Testing	Login as Admin	Confirm that the dashboard loads with access to subjects, notes, plans, and alerts.	Admin panel loaded with full control over all modules and datasets.	Passed
8.	Beta Testing	Load profile page and check note/plan privacy.	Profile displays all details. Only the user who added notes/plans can view them; no other user can access them.	User details displayed correctly. Data isolation verified: User B cannot access User A's data.	Passed

Table 6.1: Test Report

6.2 User Documentation

User documentation is an essential component of the Smart Study Planner system, helping administrators and students effectively interact with the platform. The documentation is designed to be simple, clear, and user-friendly**,** enabling users with varying technical knowledge to use the system confidently.

Contents of the User Documentation:

- **System Overview:** This section introduces the Smart Study Planner system by explaining its purpose, benefits, and key features. It helps users understand how the platform connects students with digital study, simplifies notes, plan and progress management, reduces manual errors, and provides real-time access to study resources.
- **User Guides:** The User Guides section provides step-by-step instructions to help users perform common tasks within the Smart Study Planner system. It includes guidance on registering and logging in, creating and updating student profiles, adding and managing notes, plans, study, providing notifications, and generating progress.

  FAQs (Frequently Asked Questions): The FAQs section addresses common questions users may face while using the system. This includes queries such as how to update notes, plans, how to cancel or modify an note or plan, how to reset a password, and how to view note history. Clear and simple answers help users resolve issues independently without requiring technical support.

  Troubleshooting Tips: This section provides guidance for resolving minor issues such as login errors, missing product listings, incorrect order status, or dashboard loading problems**,** allowing users to quickly fix common problems on their own.

  Contact Information: Clear instructions for reaching support, including email addresses or system administrator contacts, are provided to ensure assistance is available if needed.

CHAPTER 7

CONCLUSION AND RECOMMENDATIONS

7.1Conclusion

7.1.1 Significance of the System

The Smart Study Planner provides a digital platform for reading notes and planning. It simplifies the study plans by proving notification to meet the deadline and study hours. This improves efficiency, and maintains secure and accurate records of students, notes, plans in a more convenient and reliable.

7.1.2 Limitations of the System

It is web-based technology, so laptop or PC is required. Only authorized users can get permission to access the note. No note sharing platforms there.

7.2 Recommendations

Although the smart study planner meets its core academic tracking objectives, future upgrades can significantly elevate user experience, cognitive performance, and system reliability. Tracking, real-time notifications, advanced search and filtering, automated data backup, and a mobile application can improve user experience, security, and overall system performance. These enhancements will make the system more efficient, reliable, and scalable.

REFERENCES

A. Zimmerman, B. Kitsantas , and J. Campillo, “Self-regulated learning and academic achievement: Theoretical perspectives,” Educational Psychologist, vol.35, no. 1, pp.1-19,2000.

M. Grinberg, Flask Web Development: Developing Web Applications with Python, 2nd ed.

O'Reilly Media, 2018.

Flask Documentation — Official Flask Web Framework Documentation, Pallets Projects, 2024.[Online]. Available: https://flask.palletsprojects.com/ SmartStudy Planner — Project Proposal Page 14

PyMySQL Documentation — Python MySQL Client Library. [Online]. Available:https://pymysql.readthedocs.io/

Coursera. (n.d.). *Coursera*. https://www.coursera.org

Stack Overflow. (2018, February 23). *JavaScript search bar to display results after user input* [Online forum post]. https://stackoverflow.com/questions/48946243/javascript-search-bar-to-display-results-after-user-input

Study Track. (n.d.). *Study Track App*. https://study-track.app

Stack Overflow. (2018, February 23). *JavaScript search bar to display results after user input* [Online forum post]. https://stackoverflow.com/questions/48946243/javascript-search-bar-to-display-results-after-user-input

Study Track App. https://study-track.app

Codemy.com. (2021, March 4). *Flask database relationships - SQLAlchemy - Python flask tutorial #16* [Video]. YouTube. https://www.youtube.com/watch?v=7jKsHOZk-IE

Pallets. (n.d.). *Queries — Flask-SQLAlchemy documentation (v2.x)*. Read the Docs. https://flask-sqlalchemy.readthedocs.io/en/stable/queries

ANNEX I

Screenshots:

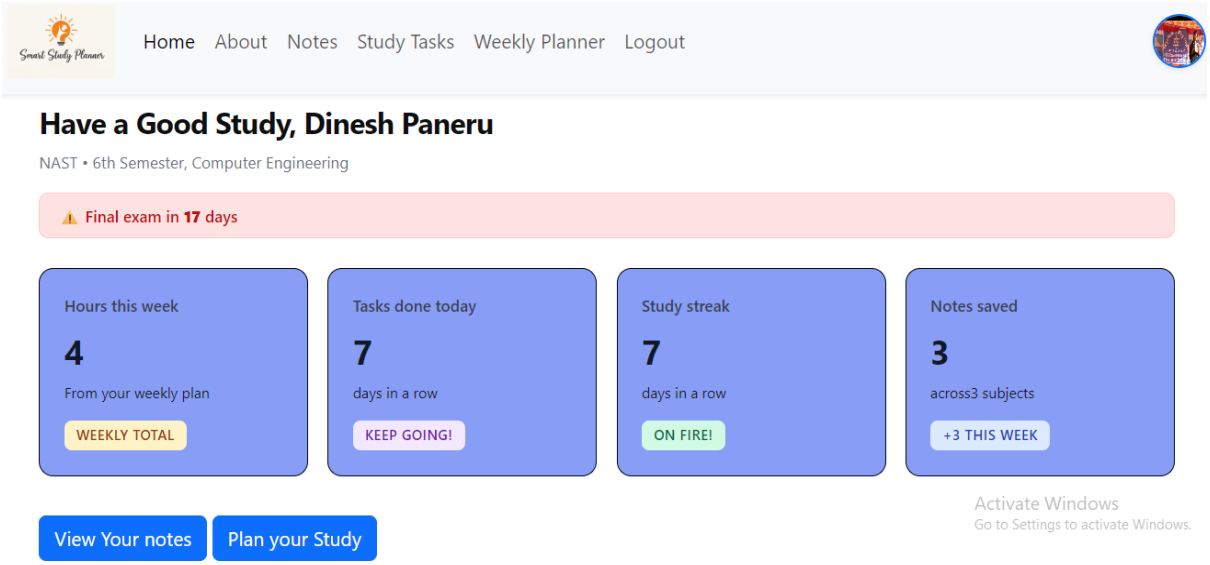


Figure: ANNEX 1.1

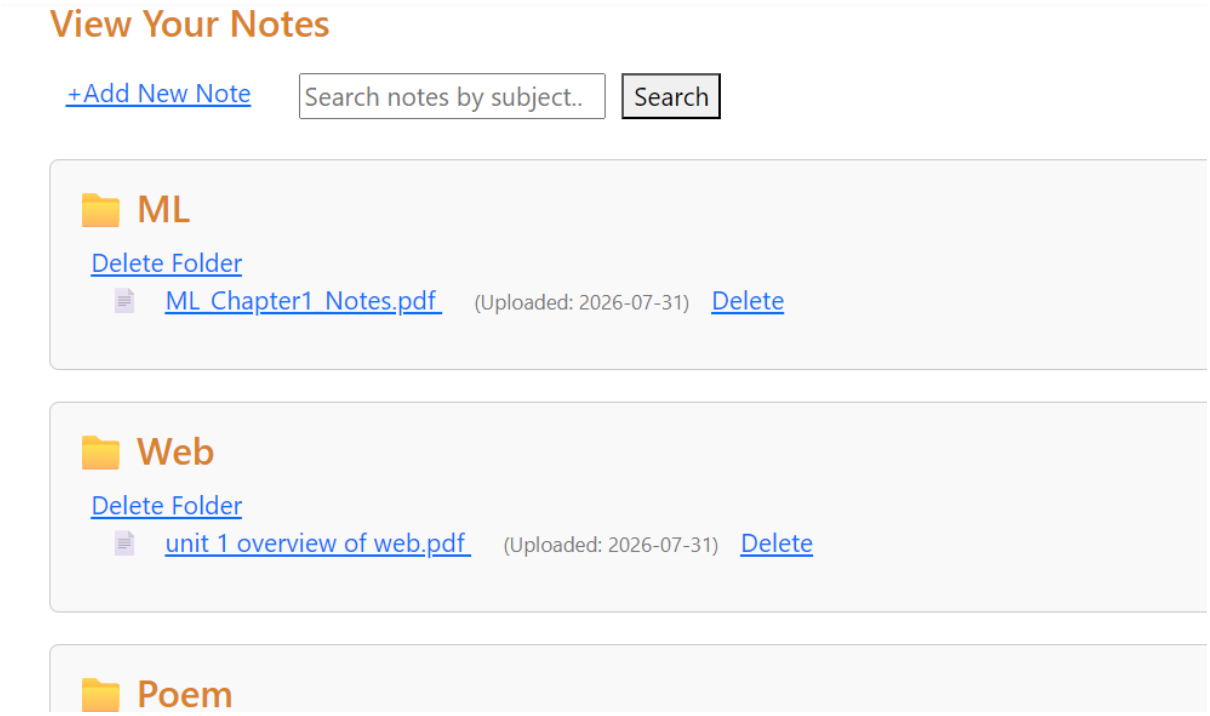


Figure: ANNEX 1.2

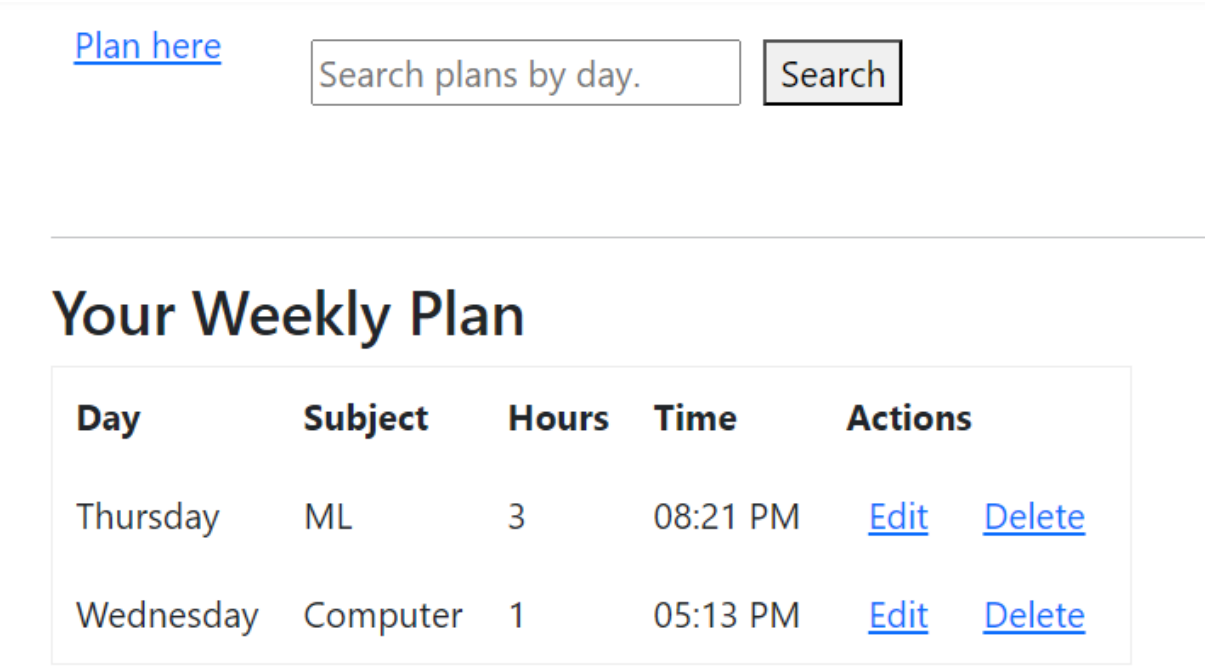


Figure: ANNEX 1.3

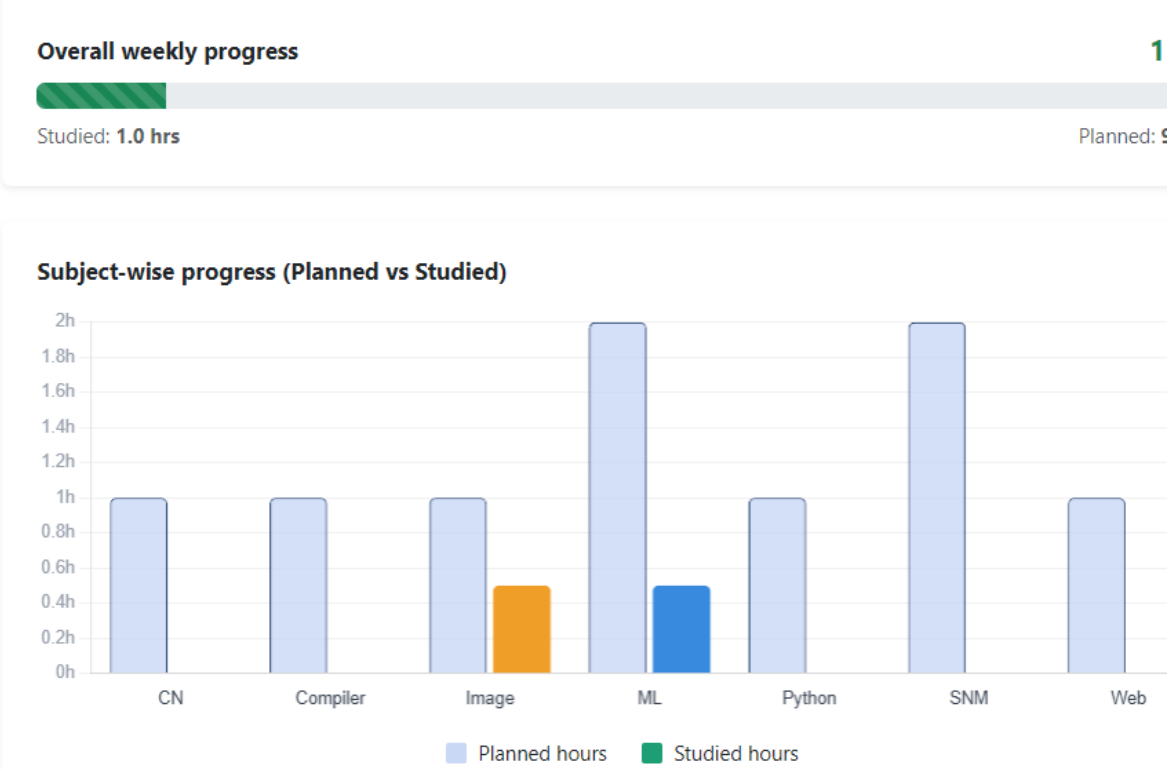


Figure: ANNEX 1.4

ANNEX II

Source code

```
@app.route("/add_notes", methods=["GET", "POST"])
def add_notes():
    if 'user_id' not in session:
        return redirect(url_for('login'))

    if request.method == "POST":

        subject = request.form.get("subject", "").strip()
        title = request.form.get("title", "").strip()
        note_content = request.form.get("note_content", "").strip()
        files = request.files.getlist("note-file")

        has_files = any(f.filename for f in files)
        has_text = bool(note_content)

        if not subject:
            flash("Subject is compulsory.", "danger")
            return redirect(url_for("add_notes"))

        if not has_files and not has_text:
            flash("Upload a file or write a text note.", "danger")
            return redirect(url_for("add_notes"))

        if has_text and not title:
```

```
    flash("Title is compulsory for text notes.", "danger")
    return redirect(url_for("add_notes"))

user_id = session["user_id"]

subject_record = Subject.query.filter_by(
    sname=subject,
    uid=user_id,
    is_deleted=False
).first()

if not subject_record:
    code = secure_filename(subject).lower() + f"_{user_id}"

    if Subject.query.filter_by(scode=code).first():
        code += str(random.randint(100, 999))

    subject_record = Subject(
        sname=subject,
        scode=code,
        uid=user_id
    )
          db.session.add(subject_record)
    db.session.commit()
folder = secure_filename(subject)
folder_path = os.path.join(app.config["UPLOAD_FOLDER"], folder)
```

```
os.makedirs(folder_path, exist_ok=True)
if has_files:
    for file in files:
            if not file or file.filename == "":
            continue
        filename = secure_filename(file.filename)
        duplicate = Note.query.filter_by(
            uid=user_id,
            sid=subject_record.sid,
            title=filename,
            is_deleted=False
        ).first()
            if duplicate:
            flash(f"{filename} already exists.", "danger")
            return redirect(url_for("add_notes"))
        save_path = os.path.join(folder_path, filename)
        file.save(save_path)
        db.session.add(
            Note(
                title=filename,
                file_path=save_path,
                uid=user_id,
                sid=subject_record.sid,
                is_deleted=False
            )
        )
```

```
if has_text:
    txt_filename = secure_filename(title).lower() + ".txt"
    duplicate = Note.query.filter_by(
        uid=user_id,
        sid=subject_record.sid,
        title=txt_filename,
        is_deleted=False
    ).first()
    if duplicate:
        flash("Text note already exists.", "danger")
        return redirect(url_for("add_notes"))
    txt_path = os.path.join(folder_path, txt_filename)
    with open(txt_path, "w", encoding="utf-8") as f:
        f.write(note_content)
    db.session.add(
        Note(
            title=txt_filename,
            file_path=txt_path,
            uid=user_id,
            sid=subject_record.sid,
            is_deleted=False
        )
    )
db.session.commit()
flash("Note added successfully.", "success")
return redirect(url_for("notes"))
```

```
    return render_template("add_notes.html")
    return redirect(url_for('notes'))


#weekplan
@app.route("/week_plan", methods=["GET", "POST"])
def week_plan():
            if 'user_id' not in session:
        return redirect(url_for('login'))
            user_id = session['user_id']
    subjects = Subject.query.filter_by(uid=user_id, is_deleted=False).all()
    search_query = request.args.get('search', '').strip()

    if  search_query:
       plans = Weekly_plan.query.filter_by(uid=user_id, is_deleted=False).filter(
            Weekly_plan.day_of_week.ilike(f"%{search_query}%")
        ).all()

       if not plans:
         flash("no plan added add first them"
    else:
        plans = Weekly_plan.query.filter_by(uid=user_id, is_deleted=False).all()
        if not plans:
            flash("No plans added yet. Add your first plan!")
    return render_template(
            "week_plan.html",
            subjects=subjects)
```

ANNEX III

Test Cases source code

```
 #login
@app.route("/login", methods=['GET', 'POST'])
def login():
    if request.method == "POST":
      email = request.form.get('email')
      password = request.form.get('password')

      user = User.query.filter_by(email=email, is_deleted=False).first()

      if user and check_password_hash(user.password, password):
            session['user_id'] = user.uid
            return redirect(url_for('dashboard'))
      else:
            flash("Invalid username or password.","danger")
            return redirect(url_for('login'))

    return render_template('login.html')

@app.route('/forgot-password', methods=['GET', 'POST'])
def forgot_password():
    if request.method == 'POST':
        email = request.form.get('email', '').strip()
        user = User.query.filter_by(email=email).first()
```

```
        if not user:
            flash("No account found with that email.", "danger")
            return redirect(url_for('forgot_password'))

        otp = str(random.randint(100000, 999999))
        user.reset_otp = otp
        user.reset_otp_expiry = datetime.utcnow() + timedelta(minutes=1)
        db.session.commit()

        msg = Message("Your Password Reset OTP", recipients=[email])
        msg.body = (
            f"Your OTP for resetting your Smart Study Planner password is: {otp}\n"
            f"This code expires in 1 minutes.\n\n"
            f"If you didn't request this, you can ignore this email."
        )
        mail.send(msg)

        session['reset_email'] = email
        flash("An OTP has been sent to your email.", "success")
        return redirect(url_for('verify_otp'))

    return render_template('forgot_password.html')

@app.route('/verify-otp', methods=['GET', 'POST'])
def verify_otp():
    email = session.get('reset_email')
```

```
    if not email:
        return redirect(url_for('forgot_password'))

    if request.method == 'POST':
        entered_otp = request.form.get('otp', '').strip()
        user = User.query.filter_by(email=email).first()

        if not user or not user.reset_otp:
            flash("Something went wrong. Please try again.", "danger")
            return redirect(url_for('forgot_password'))

        if user.reset_otp_expiry < datetime.utcnow():
            flash("OTP has expired. Please request a new one.", "danger")
            return redirect(url_for('forgot_password'))

        if str(entered_otp) != str(user.reset_otp):
         flash("Incorrect OTP. Please try again.", "danger")
         return redirect(url_for('verify_otp'))

        session['otp_verified'] = True
        flash("OTP verified. Please set a new password.", "success")
        return redirect(url_for('reset_password'))

    return render_template('verify_otp.html', email=email)

@app.route('/reset-password', methods=['GET', 'POST'])
```

```
def reset_password():
    email = session.get('reset_email')
    if not email or not session.get('otp_verified'):
        return redirect(url_for('forgot_password'))

    if request.method == 'POST':
        password = request.form.get('password', '').strip()
        confirm = request.form.get('confirm_password', '').strip()

        if len(password) < 6:
            flash("Password must be at least 6 characters.", "danger")
            return redirect(url_for('reset_password'))

        if password != confirm:
            flash("Passwords do not match.", "danger")
            return redirect(url_for('reset_password'))

        user = User.query.filter_by(email=email).first()
        user.password = generate_password_hash(password)
        user.reset_otp = None
        user.reset_otp_expiry = None
        db.session.commit()

        session.pop('reset_email', None)
        session.pop('otp_verified', None)
```

```
    flash("Password reset successful. Please log in.", "success")
    return redirect(url_for('login'))

return render_template('reset_password.html')
```